

# Bitrix24 JS SDK — SKILL.md

Перевод файла skills/b24-jssdk/SKILL.md на русский язык. Ниже — структура каталога skills/, затем — содержимое SKILL.md.

## Структура каталога

|                       |                                               |
|-----------------------|-----------------------------------------------|
| skills/               |                                               |
| ├─ index.json         | # манифест: name, description, files[]        |
| └─ b24-jssdk/         |                                               |
| ├─ SKILL.md           | # главная точка входа (этот документ)         |
| └─ references/        |                                               |
| ├─ api-surface.md     | # категоризованный индекс публичных экспортов |
| ├─ guidelines/        | # правила и решения                           |
| ├─ entry-points.md    | # B24Frame / B24Hook / B24OAuth — что выбрать |
| ├─ conventions.md     | # импорты, Result/AjaxResult, lifecycle       |
| ├─ error-handling.md  | # AjaxError, SdkError, halt-on-error          |
| ├─ rate-limiting.md   | # RestrictionManager, ParamsFactory           |
| └─ logger.md          | # LoggerBrowser, dev-mode gate                |
| └─ recipes/           | # законченные паттерны                        |
| ├─ frame-apps.md      | # initializeB24Frame + Vue/React lifecycle    |
| ├─ webhook-server.md  | # B24Hook на сервере (Node)                   |
| ├─ oauth-apps.md      | # B24OAuth + ошибки refresh-token             |
| ├─ batch-calls.md     | # callBatch / callBatchByChunk                |
| ├─ list-pagination.md | # callListMethod / fetchListMethod / ручная   |
| ├─ ui-integrations.md | # slider, dialog, parent, placement, options  |
| ├─ helper-manager.md  | # useB24Helper, профиль/валюта/опции          |
| ├─ pull-client.md     | # подписка на real-time события               |
| └─ nuxt-module.md     | # @bitrix24/b24jssdk-nuxt                     |

# Содержание SKILL.md

## Bitrix24 JS SDK

JS/TS SDK для REST API Bitrix24. Три конкретных entry point'a используют общий REST-интерфейс (**AbstractB24**) и отличаются только способом аутентификации и средой исполнения:

- **B24Frame** — работает внутри placement-iframe Bitrix24; авторизация передаётся родительским окном.
- **B24Hook** — серверный входящий webhook; секрет встроен в URL.
- **B24OAuth** — локальные приложения на OAuth; SDK сам управляет access/refresh-токенами.

Все три предоставляют одинаковые примитивы вызовов: **callMethod**, **callBatch**, **callListMethod**, **fetchListMethod**, **callBatchByChunk**. Результаты — единые объекты **Result** / **AjaxResult**.

С версии v0.4.0 поставляется только ESM + UMD (CommonJS не поддерживается). Для Nuxt-приложений используйте модуль [\[@bitrix24/b24jsdk-nuxt\]\(references/recipes/nuxt-module.md\)](#).

## Авторитетные источники по API

Этот скил отвечает на вопрос «**какой entry point и метод выбрать**» и «**как правильно всё связать**». Точные сигнатуры, параметры и формы payload'ов смотрите в первоисточниках:

- [\[packages/jsdk/README-AI.md\]\(https://github.com/bitrix24/b24jsdk/blob/main/packages/jsdk/README-AI.md\)](#) — практическое руководство по API SDK (используйте при генерации кода).
- [\[Сайт документации\]\(https://bitrix24.github.io/b24jsdk/\)](#) — публичные docs; страницы под **/reference/** повторяют типы и менеджеры.
- [\[Документация REST API Bitrix24\]\(https://apidocs.bitrix24.com/\)](#) — для имён REST-методов, параметров и форм ответов.

## Базовые правила (применять всегда)

1. **Выберите подходящий entry point** — см. [\[entry-points\]\(references/guidelines/entry-points.md\)](#). **B24Frame** — для iframe-плейсментов, **B24Hook** — для серверных скриптов, **B24OAuth** — для OAuth-приложений. Никогда не используйте **B24Hook** в браузере: это приведёт к утечке секрета webhook'a.
2. **Всегда await-те инициализатор** — **initializeB24Frame()**, **B24Hook.fromWebhookUrl()** или конструктор **B24OAuth** должны завершиться до первого **call\***-вызова.
3. **Освобождайте ресурсы при размонтировании** — вызывайте **\$b24.destroy()** при unmount компонента/страницы. Иначе Pull-клиент и слушатели сообщений будут удерживать ссылки.
4. **Только именованные импорты** — у пакета нет default-экспорта, и **sideEffects: false**. Не переименовывайте типы без необходимости — корректный tree-shaking зависит от прямых импортов.

5. **Считайте `Result` / `AjaxResult` единой формой ответа** — `getData()`, `isSuccess`, `isMore()`, `getNext()`, `getTotal()`, `getErrors()`. См.

[conventions](references/guidelines/conventions.md).

6. **Выбирайте правильную стратегию листинга** — `callListMethod` только для небольших известных наборов (< 1000), `fetchListMethod` для потоковой обработки, ручной `callMethod` + `getNext()` для произвольной пагинации. См.

[list-pagination](references/recipes/list-pagination.md).

7. **Уважайте лимиты API** — встроенный `RestrictionManager` дросселирует запросы автоматически; для enterprise-порталов `LicenseManager` подменяет параметры на более мягкие. См. [rate-limiting](references/guidelines/rate-limiting.md).

## Как пользоваться этим скилом

В зависимости от задачи **до начала кодирования** загружайте только нужные reference-файлы. Не грузите всё подряд.

### Reference-файлы

**Guidelines** — решения и соглашения:

- [entry-points](references/guidelines/entry-points.md) — матрица выбора: `B24Frame` vs `B24Hook` vs `B240Auth`, ограничения сред исполнения.
- [conventions](references/guidelines/conventions.md) — импорты, именованные экспорты, форма `Result` / `AjaxResult`, lifecycle, типичные ошибки.
- [error-handling](references/guidelines/error-handling.md) — `AjaxError`, `SdkError`, агрегирование ошибок в batch'ах, стратегии повторов.
- [rate-limiting](references/guidelines/rate-limiting.md) — `RestrictionManager`, `OperatingLimiter`, `AdaptiveDelayer`, пресеты `ParamsFactory`.
- [logger](references/guidelines/logger.md) — `LoggerBrowser.build`, dev-режим, `LoggerFactory`, дефолтный null-логгер.

**Recipes** — законченные паттерны под частые задачи:

- [frame-apps](references/recipes/frame-apps.md) — старт через `initializeB24Frame()`, lifecycle, базовые REST-вызовы.
- [webhook-server](references/recipes/webhook-server.md) — Node-сервер на `B24Hook.fromWebhookUrl()`.
- [oauth-apps](references/recipes/oauth-apps.md) — настройка `B240Auth`, обработка ошибок refresh-токена.
- [batch-calls](references/recipes/batch-calls.md) — `callBatch` (object и array формы), `callBatchByChunk`, halt-on-error.
- [list-pagination](references/recipes/list-pagination.md) — `callListMethod`, `fetchListMethod`, ручная пагинация через `callMethod` + `getNext()`.
- [ui-integrations](references/recipes/ui-integrations.md) — slider, dialog, parent, placement, options (только во фрейме).
- [helper-manager](references/recipes/helper-manager.md) — `useB24Helper()`, менеджеры profile/app/currency/license/options.
- [pull-client](references/recipes/pull-client.md) — подписка на Pull (push), channel manager.
- [nuxt-module](references/recipes/nuxt-module.md) — настройка модуля `@bitrix24/b24jsdk-nuxt`.

## Быстрая справка:

- [api-surface](references/api-surface.md) — категоризованный индекс публичных экспортов: entry points, REST-помощники, типы/enums, утилиты.

## Таблица маршрутизации

| Задача                                                          | Какие reference-файлы загрузить            |
|-----------------------------------------------------------------|--------------------------------------------|
| Сделать placement-приложение Bitrix24 (iframe)                  | entry-points, conventions, frame-apps      |
| Открывать слайдеры / диалоги / работать с UI портала            | conventions, ui-integrations               |
| Серверный скрипт / cron / Node-сервис против Bitrix24           | entry-points, conventions, webhook-server  |
| OAuth-приложение (refresh-токены)                               | entry-points, conventions, oauth-apps      |
| Использовать SDK в Nuxt-приложении                              | conventions, frame-apps, nuxt-module       |
| Получать большие объёмы данных через REST                       | conventions, list-pagination               |
| Эффективно вызвать несколько связанных методов                  | conventions, batch-calls                   |
| Показывать данные портала (текущий пользователь, валюта, опции) | conventions, helper-manager                |
| Реагировать на real-time события из портала                     | conventions, helper-manager, pull-client   |
| Обрабатывать ошибки / писать логику повторов                    | conventions, error-handling, rate-limiting |
| Тюнить дросселирование под high-volume скрипты                  | rate-limiting                              |
| Настроить логирование                                           | logger                                     |
| Общий вопрос «где экспортируется X»                             | api-surface, conventions                   |

## Установка

### Базовый SDK (любой JS/TS-проект)

```
pnpm add @bitrix24/b24jssdk
```

```
import { initializeB24Frame, B24Hook, B24OAuth } from '@bitrix24/b24jssdk'
```

Поддерживаемые версии Node: ^18, ^20 или >=22.

### Nuxt-модуль (рекомендуется для Nuxt-приложений)

```
npx nuxi module add @bitrix24/b24jssdk-nuxt
```

Модуль регистрирует только runtime-плагин — он **не** оборачивает и не переименовывает экспорты SDK. Импортируйте SDK как обычно; плагин обеспечивает SSR-безопасный доступ. См. [nuxt-module](references/recipes/nuxt-module.md).

## UMD (CDN, без бандлера)

```
<script src="https://unpkg.com/@bitrix24/b24jsdk@latest/dist/umd/index.min.js"></script>
<script>
  const $b24 = await B24Js.initializeB24Frame()
</script>
```

Глобальный объект — `window.B24Js`. UMD предназначен для статических iframe-плейсментов без сборщика — серверные `B24Hook` и OAuth-приложения должны использовать ESM.